ENEE 467: Robotics Project Laboratory
Collision-Free Kinematic Motion Planning in ROS 2
Lab 5 - Part III (Hardware, MoveIt)

This part moves the MoveIt motion planning workflow from simulation onto the real UR3e. Students will learn how motion planning requests in MoveIt are executed on physical hardware through `ros2_control`, and how robot kinematic models, semantic collision information, and robot controllers connect to form a complete motion planning pipeline. After completing this part, students should be able to:

- Use a MoveIt configuration package to interface with the motion planning framework
- Plan and execute motions via RViz on the real UR3e
- Define planning groups, poses, and collision info with SRDF
- Write collision-free joint-space planners using `pymoveit2`
- Execute trajectories via `ros2_control`
- Dynamically add objects to the planning scene

Motion planning computes collision-free, time-parameterized trajectories in joint or Cartesian space that move a robot from an initial to a goal configuration while respecting obstacles, joint limits, and dynamics. Unlike pure kinematics, it explicitly considers the robot's workspace and dynamic constraints and is thus critical for robots operating in semi-controlled or unstructured settings. Together, the three parts of this lab aim to equip you with the skills and tools to use modern motion planning frameworks (e.g., *MoveIt*) and to develop your own ROS 2-compliant motion planning programs.

Related Reading

- M. Spong et. al., "*Robot Modeling and Control*", John Wiley & Sons, Inc., 2006.
- PickNik Robotics, "*MoveIt Developer Concepts*." Available: Webpage.
- G. Russi, "*Robotics Part 32 — Trajectory Generation*." Available: Robotics Unveiled.
- S. Castro, "*How Do Robot Manipulators Move?*" Available: Robotic Sea Bass.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. Pay close attention to the lab Questions (highlighted with a box). Answer these as you proceed through the lab and include your answers in your lab writeup. There are several lab Exercises at the end of the lab manual. Complete the exercises during the lab and include your solutions in your lab writeup.

List of Questions

List of Exercises

Hardware Safety

Throughout this manual, boxes beginning with a **Question #** label contain questions you must answer as you progress through the lab:

> **Question 0.** A question that you should answer as you work through the lab manual.

⚠ **Safety and Setup Notices.** Warnings highlight critical steps for hardware safety, teach-pendant operation, and common setup pitfalls. Always read these carefully before commanding motion on the UR3e.

> ⚠ Keep clear of the robot workspace when running autonomous motion. Confirm that the pendant is in `Play` mode before proceeding.

Notes. These boxes provide background or theoretical context for the UR robot system. They are optional during lab time but useful for your report:

> UR3e kinematics are defined in a six-joint chain; MoveIt uses the URDF and joint limits to compute feasible IK solutions.

Code, commands, and output are formatted as follows:

```
ros2 launch ur_robot_driver ur_control.launch.py ...
```

```
[INFO] [launch]: UR3e driver successfully connected.
```

Files and directories appear in magenta teletype font (e.g., `~/lab7_ws/src`), while in-text code and variables use blue teletype (e.g., `object_pose`). Python type hints or XML tags appear in green (e.g., `str`, `<param>`).

1. Lab Procedure

1.1 Prerequisites

This lab assumes familiarity with URDF-based robot modeling and ROS 2, including rigid-body simulation, frame transforms, and forward/inverse kinematics. Prior exposure to motion planning (e.g., sampling-based search or polynomial trajectories) is helpful but not required.

1.2 Getting this Lab's Code

The hardware procedure lives on the `hardware` branch of the lab-5 repository. To begin, in a new terminal session, navigate to the `~/Labs` directory and clone that branch:

```
cd ~/Labs
git clone -b hardware https://github.com/ENEE467-F2026/lab-5.git lab-5-hw
cd lab-5-hw/docker
```

> All three parts of this lab run on ROS 2 Jazzy. This part adds the real UR3e arm. The UR driver and MoveIt run on Jazzy. The arm is commanded without the gripper in this part, so no change of distribution is needed.

Verify that the lab-5 image is built on your system

```
docker image ls
```

Make sure that you see the `lab-5` image in the output:

```
lab-5-image   latest    <image_id>     <N> days ago   <image_size>GB
```

Launch the lab-5 Docker container:

```
userid=$(id -u) groupid=$(id -g) docker compose -f lab-5-compose.yml run \
--rm lab-5-docker
```

You should be greeted with the container prompt `(lab-5) robot@docker-desktop: $`.

1.3 Build and Test the Required Packages

To support development of MoveIt-based programs and custom planning tools, we have provided the following seven packages:

- `ur3e_hande_planning_interfaces`: Custom task and scene management interface definitions
- `ur3e_hande_gz`: Gazebo world, models, and robot spawn configuration
- `ur3e_hande_description`: Unified URDF/XACRO and meshes for UR3e + Hand-E system
- `ur3e_hande_moveit_config`: MoveIt 2 configuration package with planners and controllers
- `ur3e_hande_moveit_scripts`: Example scripts for joint-space motion planning
- `robotiq_hande_description`: Base description package for the Robotiq Hand-E gripper
- `pymoveit2`: Instructors' fork of pymoveit2, a Python API exposing core MoveIt functionality.

From within the container, build and source your workspace:

```
cd ~/ros2_ws/
colcon build --symlink-install
source install/setup.bash
```

Then test your build by running the following command from an active container:

```
cd ~/ros2_ws/src && python3 test_docker.py
```

This should print the following output to the terminal (stop and contact your TA if it doesn't):

```
All packages for Lab 5, Part III found. Docker setup is correct.
```

1.4 Launching MoveIt 2 for the UR3e

Before controlling the robot via ROS 2, we need to start the TP UR3e program. Power ON the TP, if it isn't already, then:

1. Click **Power Off** → **ON** at the bottom-left.
2. Step out of the robot's workspace, then click the → **START** button to unlock the robot's brakes and allow it to receive programs. You should now see `Manual` on the bottom left.
3. Tap `Exit` to return to the main GUI. Then tap `Load Program`, and select `ur3e_ros.urp`.
4. Assuming packages in your workspace were built with `--symlink-install`, in a sourced `tmux` session, split panes. In the first pane, start the UR3e ROS 2 hardware driver (`ur_robot_driver`):

   ```
   ros2 launch ur_robot_driver ur_control.launch.py \
   ur_type:=ur3e robot_ip:=192.168.77.22 launch_rviz:=false
   ```

5. Click `Open` then press `Play` to start the program.

The lab computers have been set up to automatically read the configuration file passed as the `kinematics_params_file`, so you do not need to provide those yourself.

> ⚠ **NOTE:** The pendant must remain in `Play` mode for ROS 2 to control the robot. Only click the `Play` button when you are ready to command the robot via ROS 2. The robot must remain in the `IDLE` or `OFF` states at all other times.

1.4.1 Plan Motions on the UR3e using MoveIt

MoveIt 2 provides IK, collision checking, trajectory generation, and controller management for the UR3e. Turn ON the TP and start the UR3e ROS 2 driver if you haven't already. Click `Play` on the TP to ready the robot for programming.

Then, in a new `tmux` pane, launch the MoveIt bringup for the UR3e:

```
ros2 launch ur_moveit_config ur_moveit.launch.py \
ur_type:=ur3e robot_ip:=192.168.77.22
```

RViz will open and load the UR3e's URDF. Using the Move tab on the TP, jog the real robot SLOWLY, and confirm RViz mirrors the motion. If joint states do not update, MoveIt cannot plan safely. Then, maintaining a safe distance from the UR3e, using MoveIt's `MotionPlanning` RViz

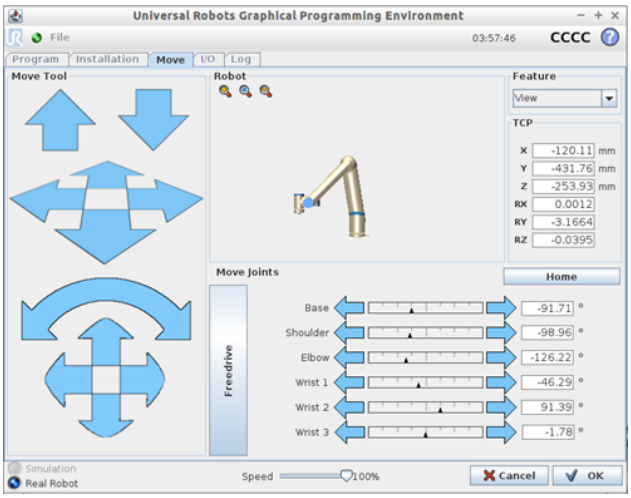

Figure 1: Robot jogging interface on the TP.

panel, plan then execute a motion to a goal you set, either by selecting a named state from the `Goal State` dropdown or by dragging the interactive marker to a new pose. You should observe the UR3e move to the selected configuration, with its state evolution mirrored in RViz.

Question 1. The UR3e ROS 2 driver publishes joint states on the topic `/joint_states`.

(a). Why must MoveIt receive these messages continuously before planning?

(b). Terminate the UR3e ROS 2 driver process and try planning to a goal from MoveIt. What failure mode, if any, do you record?

1.4.2 Programmatic Control via PyMoveIT2

Similar to prior labs, we can send programmatic goals to the robot by scripting. Open the `ur3_joint_goal.py` file in the `ur3e_hande_moveit_scripts` `ament_python` package. Ensure that both the UR3e drivers and MoveIt are active as per the commands introduced earlier. After notifying the team member(s) manning the hardware station to keep a safe distance, run:

```
ros2 run ur3e_hande_moveit_scripts ur3_joint_goal_node
```

You should observe the robot move to the default named configuration (`ur_home`). As in Part 2, you can plan to another named group state by overriding the `group_state` parameter:

```
ros2 run ur3e_hande_moveit_scripts ur3_joint_goal_node --ros-args -p \
group_state:="ur_test"
```

2. Sim-to-Real Transfer

In Part 2 you planned and executed the same motions in Gazebo. Moving the MoveIt workflow onto the real UR3e changes several things, even though the planning interface and the `pymoveit2` scripts are unchanged:

- **Kinematics and calibration**: the controller applies the robot's per-unit factory calibration, so executed poses differ slightly from the nominal model used in simulation.
- **Controller dynamics**: real joints have finite torque, friction, and tracking error, so a trajectory that executes cleanly in Gazebo may track with a small lag or settle more slowly on hardware.
- **Timing and communication**: state arrives over the network from the `ur_robot_driver`, so measured planning and execution times include communication and controller overhead that is absent in simulation.
- **Safety**: the simulator cannot harm anyone. The real robot can. Keep the emergency stop within reach, jog slowly, and clear the workspace before executing any plan.

Exercise 1 asks you to quantify some of these differences by comparing the planning and execution of the same arm motion in simulation and on hardware.

Instructions: There are two exercises in total. Both exercises must be completed. Exercise 1 is worth **40 points**. Exercise 2 is worth **40 points**.

Exercise 1. Sim-to-Real Comparison (40 pts.)

In this exercise you compare the planning and execution of the same arm motion in simulation (Part 2) and on the real UR3e. Pick one goal configuration and use it in both settings.

Deliverables:

a. Choose a single goal: either a named state from the `Goal State` dropdown or a configuration you set by dragging the interactive marker. In simulation (your Part 2 setup) and on the hardware, plan to this goal with the RViz `MotionPlanning` panel and record the planning time it reports. Execute the plan and record the execution time (wall clock).

b. Using the `Velocity Scaling` slider in the `MotionPlanning` panel, repeat (a) at scalings of 0.25, 0.15, and 0.10. Tabulate planning time and execution time for simulation versus hardware at each scaling.

c. Discuss the sim-to-real gap. Which metric changes most between simulation and hardware, and why? Relate your answer to the calibration, controller-dynamics, and timing points in the Sim-to-Real Transfer section above.

Note: Execution time on hardware is typically larger and more variable than in simulation. Planning time, computed by the same planner on the same model, should be comparable.

Exercise 2. Programmatic Joint Goals with PyMoveIt2 (40 pts.)

In this exercise you drive the arm programmatically with `ur3_joint_goal.py` (in the `ur3e_hande_moveit_scripts ament_python` package) and study synchronous versus asynchronous execution.

Deliverables:

a. Command the arm to a named group state through the `group_state` parameter:

```
ros2 run ur3e_hande_moveit_scripts ur3_joint_goal_node --ros-args \
-p group_state:="ur_test"
```

Confirm the real arm reaches the configuration. Submit the command you used and a note on what you observed.

b. The node exposes a `synchronous` parameter. With `synchronous:=true` the node waits on `wait_until_executed()` before returning. With `synchronous:=false` it returns immediately and monitors execution asynchronously. Run it both ways and describe the difference in behavior.

c. In one or two sentences, explain why a synchronous (blocking) call is the safer default when commanding a real arm.

3. Conclusion and Lab Report Instructions

After completing the lab, summarize the procedure and results into a well written lab report. Include your solutions to all of the question boxes in the lab report. Refer to the List of Questions to ensure you don't miss any. Include your full solution to the exercises in your lab report. If you wrote any Python programs during the exercises, include these files as a separate attachment, appending your team's number and last names (in alphabetical order) to the file, e.g., **Jane Doe**, **Alice Smith**, and **Richard Roe** in **Team 90** submitting their modified `main.py` would submit the file `main_Team90_Doe_Roe_Smith.py` and the report `Lab_<LabNumber>_Report_Team90_Doe_Roe_Smith.pdf`, where `<LabNumber>` is the number for that week's lab, e.g., **0**, **1**, **2**, etc. Submit your lab report along with any code to ELMS under the assignment for that week's lab **by the deadline stated on the ELMS assignment**. See the **Files** section on ELMS for a lab report template named `Lab_Report_Template.pdf`. The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	10
Exercise 1.	40
Exercise 2.	40
Total	100